ADR-001: Phase 1 Architecture Decisions

- **Status:** Accepted
- **Date:** 2026-03-06
- **Authors:** TIEVenture Team

Context

EdGame is a game-based learning analytics platform for K-12 classrooms. Phase 1 targets launch with a single game (Pulse Realms), teacher dashboards, and a core analytics pipeline. This ADR records the binding technology decisions for Phase 1.

Decisions

1. Next.js 14 App Router (not Pages Router)

Choice: Next.js 14+ with the App Router.

Rationale:

- Server Components reduce client bundle size — critical for schools with low-bandwidth networks.
- Built-in API routes (Route Handlers) eliminate the need for a separate backend service.
- Vercel-native deployment with edge functions, automatic scaling, and CI/CD.
- React Server Components enable streaming SSR for dashboard pages with heavy data.
- App Router is the stable, recommended architecture going forward; Pages Router is legacy.

Consequences:

- Team must learn React Server Component patterns (server vs. client boundaries).
- Some older React libraries may not yet support RSC.

2. KAPLAY.js as Game Engine (not Phaser)

Choice: KAPLAY.js (v3001.x) for all game environments.

Rationale:

- The original spec (spec-v1.0.md §2.2) listed Phaser 3 as the game engine. During prototyping, the team built Pulse Realms with KAPLAY.js instead.
- KAPLAY provides a simpler, more declarative API better suited to our question-gated combat mechanics.
- The existing Pulse Realms codebase (`PulseRealms/main.js`) imports from `kaplay@3001.0.19` and all game systems (telemetry, combat, question engine, progression) are built on KAPLAY primitives.
- Switching back to Phaser would require a full rewrite of the working prototype with no clear benefit.

Decision: Adopt KAPLAY.js as the standard game engine. Update the spec to reflect this. Future games (Concept Cascade, Lab Explorer, Knowledge Quest, Survival Equation) will also use KAPLAY.

Consequences:

- Smaller community than Phaser — fewer third-party plugins available.
- Team owns more low-level game infrastructure (asset loading, scene management).

3. SurrealDB as Database

Choice: SurrealDB as the primary database.

Rationale:

- Multi-model database (document, graph, relational) — fits our mixed data needs (structured schemas for users/classes, semi-structured JSONB-like payloads for game events, graph relations for class membership).
- Built-in permissions system (`DEFINE TABLE ... PERMISSIONS`) provides data isolation between schools/teachers without application-level authorization code.
- SurrealQL is expressive and supports record links, subqueries, and computed fields natively.
- Easy to deploy as a single service on Railway — no managed service vendor lock-in.
- Built-in functions and live queries for real-time dashboard updates.
- Authentication handled via NextAuth.js at the application layer, with SurrealDB storing user records.

Consequences:

- Newer database with a smaller ecosystem than PostgreSQL — fewer ORMs, tools, and community resources.
- Auth is handled at the Next.js layer (NextAuth.js) rather than database-level.
- Team must learn SurrealQL syntax (similar to SQL but with differences).

4. Railway for Hosting

Choice: Railway for all web infrastructure and database hosting.

Rationale:

- Simple deployment from GitHub with automatic builds.
- Supports Next.js natively (Nixpacks builder auto-detects).
- Can host both the Next.js app and SurrealDB as separate services in one Railway project.
- Persistent volumes for SurrealDB data.
- No serverless cold-start latency — runs as a persistent process.
- Simpler configuration than Vercel for monorepo + database co-hosting.

Consequences:

- No edge functions — API routes run from a single region (acceptable for Phase 1).
- Must configure Railway services manually (Next.js service + SurrealDB service).
- Long-running jobs (daily rollups) can run as cron jobs within the Next.js service or as SurrealDB functions.

5. pnpm Monorepo with Turborepo

Choice: pnpm workspaces managed by Turborepo.

Rationale:

- Multiple packages need to share code: `apps/web` (Next.js dashboard), `apps/games` (KAPLAY game environments), `packages/shared` (types, utilities, constants), `packages/ui` (shadcn/ui component library).
- pnpm provides strict dependency isolation (no phantom dependencies) and efficient disk usage via content-addressable storage.
- Turborepo provides incremental builds, remote caching, and parallel task execution.
- Both are Vercel-maintained, ensuring long-term compatibility.

Consequences:

- Slightly higher setup complexity vs. a single-package repo.
- CI must be configured for Turborepo caching.

6. Phased Scaling Strategy

Choice: Three-phase infrastructure scaling tied to user milestones.

Phase	Users	Infrastructure	Key Changes
Phase 1 (Launch)	0–5,000	Railway (Next.js + SurrealDB)	Single SurrealDB instance, client-side telemetry batching, daily metric rollups via cron
Phase 2 (5K–50K)	5,000–50,000	Railway Pro + dedicated SurrealDB	Separate analytics namespace, event queue, Redis caching for dashboards
Phase 3 (50K+)	50,000+	Multi-service Railway or dedicated infrastructure	SurrealDB cluster, CDN edge caching for game assets, data warehouse for longitudinal analytics

Rationale:

- Avoids premature optimization — Phase 1 keeps infrastructure simple and cheap.
- Clear trigger points for scaling decisions (not time-based, load-based).
- Each phase has a concrete migration plan, not just "we'll figure it out."

Consequences:

- Phase 1 architecture must be designed so that Phase 2 migrations are non-breaking (e.g., event schema must support future queue-based ingestion).

References

- `docs_markdown/spec-v1.0.md` — Original specification
- `PulseRealms/main.js` — KAPLAY import and game bootstrap
- `PulseRealms/src/systems/telemetry.js` — Event format used in prototype
- `docs/architecture/data-model.surql` — Phase 1 database schema (SurrealQL)
- `docs/architecture/api-routes.md` — Phase 1 API route inventory
- `docs/architecture/monorepo-structure.md` — Directory layout